



Tagging every issue for self-service & visit planning

Summary of the classification requirement and our implementation plan, with four recommended refinements. Prepared by Mihir Sethi · 19 July 2026.

1 · What was asked

As the team begins populating the tool this quarter (Nagender creating content, Priyanka testing), every issue captured in the system should carry **mandatory classification tags on a 2×2 matrix**, assigned at the moment content is created:

Axis 1 — Where it can be solved

Can this be resolved without anyone travelling to the plant, or does it require on-site involvement?

Axis 2 — Who can solve it

Can a less-technical person complete it, or does it need a specialist?

The three payoffs identified:

1. **Self-service accountability** — compare what a user *should* have been able to solve against when they raised a ticket (flag it, or potentially lock ticket-raising until the doable steps are done) → fewer site visits.
2. **Engineer planning** — a probabilistic view of how much incoming work genuinely requires on-site specialists.
3. **A compounding "root database"** — cheap tags today that unlock analytics for years.

2 · How it maps to the tool

Issues live in the tool as **diagnostic flows** (step-by-step trees the Dr. Paani assistant walks users through), generated by AI from approved documents and reviewed by an admin before going live. Classification slots into that existing pipeline with **near-zero extra typing**:

- **AI proposes tags for every step** it drafts — it is already reading the procedure; classifying "clean the probe" is straightforward for it.
- **The admin confirms with tap-to-toggle chips** in the existing flow-review screen.
- **Approval is blocked until classification is confirmed** — "mandatory," enforced at the one gate all content already passes through.
- **Users get the matching attribute** — a technical level on each profile (the Priyanka-vs-Neeraj distinction), so expectations can be computed per user.
- An issue's overall classification is **derived automatically from its steps** (if any step needs a visit, the issue does).

3 · Four recommended refinements

We pressure-tested the design before building. The core idea holds; these adjustments protect the quality of the dataset it produces.

a. Sharper axis labels — "Visit needed?" instead of "Remote?"

The person using Dr. Paani is usually standing at the sensor, so "remote" is ambiguous — is *clean the probe* remote? It's physical work, yet needs no visit. Since the goal is reducing site visits, we propose the labels "**No visit needed / Visit required**" and "**Anyone / Specialist**", each with a one-line definition shown at the point of tagging. Consistent interpretation is what makes the analytics trustworthy.

b. Mandatory at the issue level; AI-tagged at the step level

Requiring a human decision on every step of every flow (~12 micro-decisions per approval) invites rubber-stamping — which *looks* like data but isn't. Instead: AI tags every step automatically (kept, editable), and the reviewer must confirm **two chips per issue**. Planning analytics need exactly this level; step-level precision can be promoted later if the data shows we need it.

c. Flag, don't lock, ticket-raising (v1)

Locking has two failure modes: a mis-tagged "you can do this yourself" in front of a down compliance sensor is an operational incident; and if clicking through steps unlocks the ticket button, people will click without doing — corrupting the very data this system exists to collect. A **flag on the ticket** ("raised with 3 self-serviceable steps not attempted") delivers the same accountability data safely. Lock can become a config toggle later once tag quality is proven.

d. Timing: capture now is cheap — and backfill is a safety net, not a crisis

Because content here is structured and the AI classifier is already in the pipeline, existing flows can be **backfilled in a single batch run + review pass** if anything is created before this ships. We're still building capture-at-creation first (it's nearly free), but population starting before PO lands would lose nothing.

4 • Two additions we recommend

- **Log step-level outcomes** (which steps users complete or abandon). Tags say what *should* be self-serviceable; outcomes show what *actually* is. Together they answer the planning question better than either alone — and empirical data corrects any mis-tags over time.
- **Aggregate-first analytics**. Start with team-level views (visit-required rate, specialist demand, self-service gap). Per-person expectation-vs-actual reporting is possible on the same data, but deserves an explicit, announced decision — and tester accounts (e.g. Priyanka's) must be excluded so testing activity doesn't read as failed self-service.

5 • Delivery plan

PHASE

SCOPE

PURPOSE

P0 · this week	Classification schema · AI-proposed step tags · mandatory issue-level confirmation at approval · technical level on user profiles	In place before population begins — every flow carries tags from day one
P1	Step-outcome logging · tags visible in chat, with early "this needs a specialist visit" hand-off	Starts accumulating the dataset
P2	Ticket flag · Insights planning panel (visit-required rate, specialist demand — aggregate)	The payoffs, computed on P0/P1 data

Even if P1/P2 shift, P0 guarantees the "root database" grows correctly from the first document the team adds.

6 · Open questions

1. Confirm **flag-not-lock** for v1 ticket-raising (our recommendation, §3c)?
2. Are the axis labels "**No visit needed / Visit required**" and "**Anyone / Specialist**" acceptable, or is there preferred terminology?